Sistema de Gestion de Base de Datos para una Clinica

Evaluacion Parcial N° 2 – Procedimientos, Funciones, Packages y Triggers

BDY1103 – Taller de Base de Datos · Sebastian Antipan · DUOC UC

Versión con código completo · incluye 8 láminas de respaldo (R1-R8)

Contexto y problema

Una clínica que necesita datos confiables

Agenda, stock, reportes y trazabilidad: la EP2 los resuelve dentro de la base

Agenda sin control.
Las citas se agendan a mano y pueden quedar con un paciente o un médico que no existe.

Stock sin respaldo.
Cada receta debe descontar medicamentos; si la aplicación lo olvida, el inventario queda mal.

Reportes dispersos.
Contar pacientes, médicos y citas obliga a repetir la misma consulta en cada programa.

Sin trazabilidad.
No queda un registro claro de cuándo se insertó información en la agenda.

PARTE 1

Procedimientos y funciones

Con y sin parámetros · usados en sentencias SQL y en otros programas PL/SQL

Procedimiento vs. función: la diferencia clave

Definición, características y ventajas

PROCEDIMIENTO

- Ejecuta una ACCIÓN (insertar, validar, reportar).
- Puede recibir parámetros (IN) o venir sin ellos.
- NO devuelve valor con RETURN: informa por pantalla o insertando datos.
- Ventaja: centraliza acciones y se reutiliza sin copiar código.
- Desventaja: sus resultados no se usan dentro de un SELECT.

FUNCIÓN

- Calcula y DEVUELVE un valor con RETURN obligatorio.
- Con o sin parámetros; también puede recibir lo mismo que un procedimiento.
- Se usa DENTRO de sentencias SQL (SELECT, WHERE) y en otros bloques.
- Ventaja: encapsula cálculos reutilizables y masivos.
- Desventaja: debe retornar en todos los caminos, incluso con excepciones.

El reporte general de la clínica (120 pacientes, 10 médicos, citas y stock)

Procedimiento sin parámetros: sp_resumen_clinica

Procesa TODA la información sin pedir nada de entrada

CÓDIGO

```
CREATE OR REPLACE PROCEDURE sp_resumen_clinica IS
    v_total_pacientes    NUMBER;
    v_total_medicos      NUMBER;
    v_total_citas        NUMBER;
    v_stock_medicamentos NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_total_pacientes FROM paciente;
    SELECT COUNT(*) INTO v_total_medicos   FROM medico;
    SELECT COUNT(*) INTO v_total_citas     FROM cita;
    SELECT NVL(SUM(stock), 0) INTO v_stock_medicamentos FROM medicamento;
    DBMS_OUTPUT.PUT_LINE('===== RESUMEN GENERAL DE LA CLINICA =====');
    DBMS_OUTPUT.PUT_LINE('Pacientes registrados    : ' || v_total_pacientes);
    DBMS_OUTPUT.PUT_LINE('Medicos registrados      : ' || v_total_medicos);
    DBMS_OUTPUT.PUT_LINE('Citas registradas        : ' || v_total_citas);
    DBMS_OUTPUT.PUT_LINE('Unidades de medicamentos : ' || v_stock_medicamentos);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error inesperado: ' || SQLERRM);
END sp_resumen_clinica;
/
```

SALIDA

```
===== RESUMEN GENERAL DE LA
CLINICA =====
Pacientes registrados    : 120
Medicos registrados      : 10
Citas registradas        : 243
Unidades de medicamentos : 3489
```

- **En SQL:** CALL sp_resumen_clinica();
- **En otro programa PL/SQL:** BEGIN sp_resumen_clinica; END; — la misma lógica, una sola vez escrita.

Justificación: evita citas con pacientes o médicos inexistentes

Procedimiento con parámetros: sp_registrar_cita

Valida paciente y médico para agendar con cualquier dato

CÓDIGO

Código completo → R1

```
CREATE OR REPLACE PROCEDURE sp_registrar_cita (
    p_id_paciente    IN paciente.id_paciente%TYPE,
    p_id_medico      IN medico.id_medico%TYPE,
    p_id_consultorio IN consultorio.id_consultorio%TYPE,
    p_fecha_cita     IN DATE,
    p_hora_cita      IN VARCHAR2
) IS
    v_existe_paciente NUMBER;
    v_existe_medico   NUMBER;
    v_nuevo_id_cita   NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_existe_paciente
    FROM paciente WHERE id_paciente = p_id_paciente;
    -- valida ambos; si existen, INSERT + COMMIT
    DBMS_OUTPUT.PUT_LINE('Cita #' || v_nuevo_id_cita
        || ' registrada correctamente.');
EXCEPTION
    WHEN OTHERS THEN ROLLBACK;
END sp_registrar_cita;
/
```

SALIDA

```
sp_registrar_cita(
  p_id_paciente => 1, p_id_medico => 2,
  p_id_consultorio => 3,
  p_fecha_cita => SYSDATE + 7,
  p_hora_cita => '10:30');
Cita #242 registrada correctamente.
```

- **Parámetros:** la MISMA lógica sirve para el paciente 1 con el médico 2 hoy y para cualquier otro mañana.
- **Regla de negocio:** si el paciente o el médico no existen, la cita no se inserta.

Cuenta todos los pacientes sin recibir datos

Función sin parámetros: fn_total_pacientes

fn_total_pacientes: no recibe datos y devuelve el total

CÓDIGO

Código completo → R3 1/2

```
CREATE OR REPLACE FUNCTION fn_total_pacientes
RETURN NUMBER IS
    v_total NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_total FROM paciente;
    RETURN v_total;
EXCEPTION
    WHEN OTHERS THEN RETURN -1;
END fn_total_pacientes;
/
```

SALIDA

```
TOTAL_PACIENTES_DE_LA_CLINICA
-----------------------------
                          120
```

- Sin parámetros: no necesita entrada.
- Devuelve el total de pacientes: 120.

Calcula la edad de un paciente a partir de su id

Función con parámetros: fn_calcular_edad

fn_calcular_edad: recibe el id y devuelve la edad

CÓDIGO

Código completo → R3 2/2

```
CREATE OR REPLACE FUNCTION fn_calcular_edad (
    p_id_paciente IN paciente.id_paciente%TYPE
) RETURN NUMBER IS
    v_fecha_nac paciente.fecha_nacimiento%TYPE;
BEGIN
    SELECT fecha_nacimiento INTO v_fecha_nac
    FROM paciente WHERE id_paciente = p_id_paciente;
    RETURN TRUNC(MONTHS_BETWEEN(SYSDATE, v_fecha_nac) / 12);
EXCEPTION
    WHEN NO_DATA_FOUND THEN RETURN NULL;
END fn_calcular_edad;
/
```

SALIDA

```
ID_PACIENTE NOMBRE_COMPLETO        EDAD
----------- -------------------- -----
  1         Emilia Diaz           18
  2         Vicente Contreras     48
  ... (hasta los 10 primeros)
```

- Con parámetros: recibe el id de un paciente.
- Devuelve su edad y se usa dentro de un SELECT.
- Ambas se pueden llamar desde SQL y desde PL/SQL.

PARTE 2

Packages

Construcción pública y privada, y cómo el package integra la solución

Package: parte pública y parte privada

Lo público se comparte; lo privado queda adentro

ESPECIFICACIÓN (pública)

- **fn_citas_medico** — cuenta las citas de un médico (cualquiera puede llamarla, hasta un SELECT).
- **sp_mostrar_usos** — muestra el contador interno de la sesión.
- Aquí declara QUÉ existe, no CÓMO funciona.

CUERPO (público + privado)

- **v_usos** — variable interna que cuenta los usos (solo el package la ve).
- **fn_medico_existe** — función de apoyo para validar: NO está en la spec, nadie de afuera la llama.
- **Las públicas** se implementan aquí; las privadas solo existen aquí.
- **Ventaja:** encapsulamiento y una única puerta de entrada al reporte.

Uso masivo en SQL y regla interna protegida

Demostración del package

Una función pública reutiliza una privada para validar

CÓDIGO

Código completo → R4-R5

```
PACKAGE BODY pkg_reportes_clinica IS
    -- PRIVADO: contador interno de sesion
    v_usos NUMBER := 0;

    -- PRIVADO: funcion apoyo (NO esta en la spec)
    FUNCTION fn_medico_existe(p_id_medico IN medico.id_medico%TYPE) RETURN NUMBER IS
        v_cuenta NUMBER;
    BEGIN
        SELECT COUNT(*) INTO v_cuenta FROM medico
        WHERE id_medico = p_id_medico;
        RETURN v_cuenta;
    END fn_medico_existe;
    -- PUBLICO: implementa la funcion de la spec
    FUNCTION fn_citas_medico(p_id_medico IN medico.id_medico%TYPE) RETURN NUMBER IS
        v_total NUMBER;
    BEGIN
        v_usos := v_usos + 1;
        IF fn_medico_existe(p_id_medico) = 0 THEN
            RETURN -1;   -- regla interna
        END IF;
        SELECT COUNT(*) INTO v_total FROM cita
        WHERE id_medico = p_id_medico;
        RETURN v_total;
    END;
END pkg_reportes_clinica;
```

SALIDA

```
MEDICO NOMBRE_MEDICO        CITAS
------ -------------------  -----
1      Andrea Soto            29
2      Bastian Rojas          22
3      Carla Fuentes          28
...    (10 filas en total)

SELECT fn_citas_medico(9999) FROM dual;
-> -1  (medico inexistente)
```

SALIDA

```
BEGIN
  v := pkg.fn_citas_medico(2);
  pkg.sp_mostrar_usos;
END;
-- Este package se ha usado
-- 12 veces en esta sesion.
```

PARTE 3

Triggers

Nivel de fila y nivel de sentencia — automatización que nadie llama

trg_descontar_stock (fila) y trg_bitacora_cita (sentencia)

Trigger de fila vs. trigger de sentencia

Reglas que la base ejecuta sola

NIVEL DE FILA

- FOR EACH ROW: una vez por CADA fila afectada.
- Lee los datos de cada fila con :NEW y :OLD.
- **En la clínica:** cada detalle de receta descuenta 1 del stock de su medicamento (10 detalles = 10 descuentos).
- **Propósito:** cuidar el inventario sin que nadie lo consulte a mano.

NIVEL DE SENTENCIA

- SIN FOR EACH ROW: UNA sola vez por sentencia.
- Aunque la sentencia inserte 1 fila o 500 de una vez.
- **En la clínica:** cada INSERT a cita deja UN registro en bitacora_citas, no uno por fila.
- **Propósito:** trazabilidad limpia de los accesos a la agenda.

Código del trigger de fila → R6

Trigger de fila: trg_descontar_stock

Cada INSERT en receta_detalle descuenta 1 del stock

CÓDIGO

Código completo → R6

```
CREATE OR REPLACE TRIGGER trg_descontar_stock
AFTER INSERT ON receta_detalle
FOR EACH ROW
DECLARE
    v_stock_actual medicamento.stock%TYPE;
BEGIN
    SELECT stock INTO v_stock_actual
    FROM medicamento
    WHERE id_medicamento = :NEW.id_medicamento;
    IF v_stock_actual > 0 THEN
        UPDATE medicamento
        SET stock = stock - 1
        WHERE id_medicamento = :NEW.id_medicamento;
    ELSE
        DBMS_OUTPUT.PUT_LINE('Aviso: el medicamento ' || :NEW.id_medicamento
            || ' ya no tiene stock disponible.');
    END IF;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error en trigger trg_descontar_stock: '
            || SQLERRM);
END trg_descontar_stock;
/
```

SALIDA

```
Prueba:
INSERT de 1 detalle
-> el stock del medicamento
   baja exactamente 1

(10 detalles = 10 descuentos)
```

- FOR EACH ROW: se dispara una vez por cada fila insertada.
- Usa :NEW para leer el id del medicamento de la fila nueva.
- Si el stock llega a 0, avisa y no descuenta.

Trigger de sentencia: trg_bitacora_cita

Sin FOR EACH ROW: 1 registro por sentencia

CÓDIGO

Código completo → R7

```
CREATE OR REPLACE TRIGGER trg_bitacora_cita
AFTER INSERT ON cita
DECLARE
    v_nuevo_id NUMBER;
BEGIN
    SELECT NVL(MAX(id_bitacora), 0) + 1 INTO v_nuevo_id FROM bitacora_citas;

    INSERT INTO bitacora_citas (
        id_bitacora, usuario_bd, detalle
    ) VALUES (
        v_nuevo_id,
        USER,
        'Se ejecuto una insercion en la tabla CITA '
        || '(nivel de sentencia)'
    );

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error en trigger trg_bitacora_cita: '
            || SQLERRM);
END trg_bitacora_cita;
/
```

SALIDA

```
ID  FECHA               DETALLE
--- ------------------- ------------------
1   27-10-2026 16:55    Se ejecuto una
                        insercion en la tabla
                        CITA (nivel de sentencia)
```

- SIN FOR EACH ROW: se dispara una sola vez por sentencia.
- Aunque la sentencia inserte 1 fila o 500.

Demostración: 3 citas en una sola sentencia

Validación del trigger de sentencia

3 filas insertadas a la vez, y solo 1 registro en la bitácora

Código completo → R8

CÓDIGO

```
DECLARE
    v_base NUMBER;
BEGIN
    SELECT NVL(MAX(id_cita), 0) INTO v_base FROM cita;

    INSERT INTO cita (
        id_cita, id_paciente, id_medico, id_consultorio,
        fecha_cita, hora_cita, estado
    )
    SELECT v_base + ROWNUM,
           id_paciente,
           1,
           1,
           SYSDATE + 7,
           '09:00',
           'PROGRAMADA'
    FROM paciente
    WHERE ROWNUM <= 3;

    DBMS_OUTPUT.PUT_LINE('La sentencia inserto ' || SQL%ROWCOUNT
        || ' citas de una sola vez.');
    COMMIT;
END;
/
```

SALIDA

```
ID  FECHA               DETALLE
--- ------------------- ------------------
1   27-10-2026 16:55    Se ejecuto una
                        insercion en la tabla
                        CITA (nivel de sentencia)

-> 3 citas insertadas, SOLO 1 registro
```

- El trigger de sentencia se disparó 1 vez, no 3.
- Por eso la bitácora tiene 1 registro y no 3.

Resumen del aporte de cada objeto

Conclusión

La base de datos ahora procesa y genera información

Procedimientos
— acciones del negocio centralizadas: registrar citas y el resumen masivo, sin repetir código.

Funciones
— cálculos reutilizables (edad, totales) viviendo dentro del propio motor.

Packages
— una única y segura puerta de entrada al reporte: interfaz pública, lógica protegida.

Triggers
— reglas de inventario y trazabilidad que la base garantiza por sí sola.

Impacto
— la información está validada, automatizada y auditable para toda la clínica.

← Volver a la lámina 17

RESPALDO

Código completo

Para responder cuando el profe pida el detalle

R1-R2 sp_registrar_cita

R3 fn_total_pacientes · fn_calcular_edad (1/2 y 2/2)

R4-R5 pkg_reportes_clinica

R6 trg_descontar_stock (fila)

R7 trg_bitacora_cita (sentencia)

R8 validación del trigger

No se presentan en los 12 min: se usan si el profe pide el detalle.

← Volver a la lámina 6

sp_registrar_cita · 1/2

Firma, variables y validación del paciente

CÓDIGO COMPLETO

```
CREATE OR REPLACE PROCEDURE sp_registrar_cita (
    p_id_paciente     IN paciente.id_paciente%TYPE,
    p_id_medico       IN medico.id_medico%TYPE,
    p_id_consultorio  IN consultorio.id_consultorio%TYPE,
    p_fecha_cita      IN DATE,
    p_hora_cita       IN VARCHAR2
) IS

    v_existe_paciente NUMBER;
    v_existe_medico   NUMBER;
    v_nuevo_id_cita   NUMBER;

BEGIN
    SELECT COUNT(*) INTO v_existe_paciente
    FROM paciente WHERE id_paciente = p_id_paciente;

    SELECT COUNT(*) INTO v_existe_medico
    FROM medico WHERE id_medico = p_id_medico;

    IF v_existe_paciente = 0 THEN
        DBMS_OUTPUT.PUT_LINE('No se puede registrar la cita: el paciente '
            || p_id_paciente || ' no existe.');
        RETURN;
    END IF;
```

← Volver a la lámina 6

sp_registrar_cita · 2/2

Validación del médico, INSERT, COMMIT y error

CÓDIGO COMPLETO

```
    IF v_existe_medico = 0 THEN
        DBMS_OUTPUT.PUT_LINE('No se puede registrar la cita: el medico '
            || p_id_medico || ' no existe.');
        RETURN;
    END IF;

    SELECT NVL(MAX(id_cita), 0) + 1 INTO v_nuevo_id_cita FROM cita;

    INSERT INTO cita (
        id_cita, id_paciente, id_medico, id_consultorio,
        fecha_cita, hora_cita, estado
    ) VALUES (
        v_nuevo_id_cita, p_id_paciente, p_id_medico, p_id_consultorio,
        p_fecha_cita, p_hora_cita, 'PROGRAMADA'
    );

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Cita #' || v_nuevo_id_cita || ' registrada correctamente '
        || 'para el paciente ' || p_id_paciente
        || ' con el medico ' || p_id_medico || '.');

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error al registrar la cita: ' || SQLERRM);
END sp_registrar_cita;
/
```

← Volver a la lámina 7

fn_total_pacientes

Función sin parámetros y uso en SQL

CÓDIGO COMPLETO

```
CREATE OR REPLACE FUNCTION fn_total_pacientes RETURN NUMBER IS
    v_total NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_total FROM paciente;
    RETURN v_total;
EXCEPTION
    WHEN OTHERS THEN
        RETURN -1;
END fn_total_pacientes;
/

-- Uso en SQL
SELECT fn_total_pacientes AS total_pacientes_de_la_clinica FROM dual;
```

← Volver a la lámina 7

fn_calcular_edad

Función con parámetros y uso en SQL

CÓDIGO COMPLETO

```
CREATE OR REPLACE FUNCTION fn_calcular_edad (
    p_id_paciente IN paciente.id_paciente%TYPE
) RETURN NUMBER IS
    v_fecha_nac  paciente.fecha_nacimiento%TYPE;
BEGIN
    SELECT fecha_nacimiento
    INTO v_fecha_nac
    FROM paciente
    WHERE id_paciente = p_id_paciente;
    RETURN TRUNC(MONTHS_BETWEEN(SYSDATE, v_fecha_nac) / 12);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN NULL;
END fn_calcular_edad;
/

-- Uso en SQL
SELECT fn_calcular_edad(id_paciente) AS edad
FROM paciente
WHERE ROWNUM <= 10;
```

← Volver a la lámina 11

pkg_reportes_clinica · 1/2

Especificación pública y parte privada

CÓDIGO COMPLETO

```
CREATE OR REPLACE PACKAGE pkg_reportes_clinica IS
    FUNCTION fn_citas_medico (
        p_id_medico IN medico.id_medico%TYPE
    ) RETURN NUMBER;
    PROCEDURE sp_mostrar_usos;
END pkg_reportes_clinica;
/

CREATE OR REPLACE PACKAGE BODY pkg_reportes_clinica IS
    v_usos NUMBER := 0;
    FUNCTION fn_medico_existe (
        p_id_medico IN medico.id_medico%TYPE
    ) RETURN NUMBER IS
        v_cuenta NUMBER;
    BEGIN
        SELECT COUNT(*) INTO v_cuenta
        FROM medico
        WHERE id_medico = p_id_medico;
        RETURN v_cuenta;
    END fn_medico_existe;
```

← Volver a la lámina 11

pkg_reportes_clinica · 2/2

Parte pública, EXCEPTION -2 y contador de usos

CÓDIGO COMPLETO

```
    FUNCTION fn_citas_medico (
        p_id_medico IN medico.id_medico%TYPE
    ) RETURN NUMBER IS
        v_total NUMBER;
    BEGIN
        v_usos := v_usos + 1;
        IF fn_medico_existe(p_id_medico) = 0 THEN
            RETURN -1;
        END IF;
        SELECT COUNT(*) INTO v_total
        FROM cita
        WHERE id_medico = p_id_medico;
        RETURN v_total;
    EXCEPTION
        WHEN OTHERS THEN
            RETURN -2;
    END fn_citas_medico;
    PROCEDURE sp_mostrar_usos IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Este package se ha usado ' || v_usos
            || ' veces en esta sesion.');
    END sp_mostrar_usos;
END pkg_reportes_clinica;
/
```

← Volver a la lámina 14

trg_descontar_stock

Trigger de fila: una vez por detalle de receta

CÓDIGO COMPLETO

```
CREATE OR REPLACE TRIGGER trg_descontar_stock
AFTER INSERT ON receta_detalle
FOR EACH ROW
DECLARE
    v_stock_actual medicamento.stock%TYPE;
BEGIN
    SELECT stock INTO v_stock_actual
    FROM medicamento
    WHERE id_medicamento = :NEW.id_medicamento;
    IF v_stock_actual > 0 THEN
        UPDATE medicamento
        SET stock = stock - 1
        WHERE id_medicamento = :NEW.id_medicamento;
    ELSE
        DBMS_OUTPUT.PUT_LINE('Aviso: el medicamento ' || :NEW.id_medicamento
            || ' ya no tiene stock disponible.');
    END IF;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error en trigger trg_descontar_stock: ' || SQLERRM);
END trg_descontar_stock;
/
```

← Volver a la lámina 15

trg_bitacora_cita

Trigger de sentencia: una vez por INSERT

CÓDIGO COMPLETO

```
CREATE OR REPLACE TRIGGER trg_bitacora_cita
AFTER INSERT ON cita
DECLARE
    v_nuevo_id NUMBER;
BEGIN
    SELECT NVL(MAX(id_bitacora), 0) + 1 INTO v_nuevo_id FROM bitacora_citas;
    INSERT INTO bitacora_citas (
        id_bitacora, usuario_bd, detalle
    ) VALUES (
        v_nuevo_id,
        USER,
        'Se ejecuto una insercion en la tabla CITA '
        || '(nivel de sentencia)'
    );
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error en trigger trg_bitacora_cita: ' || SQLERRM);
END trg_bitacora_cita;
/
```

← Volver a la lámina 16

Validación del trigger

3 citas en una sentencia → 1 registro (27-10-2026 16:55)

CÓDIGO COMPLETO

```
DECLARE
    v_base NUMBER;
BEGIN
    SELECT NVL(MAX(id_cita), 0) INTO v_base FROM cita;
    INSERT INTO cita (
        id_cita, id_paciente, id_medico, id_consultorio,
        fecha_cita, hora_cita, estado
    )
    SELECT v_base + ROWNUM,
           id_paciente,
           1,
           1,
           SYSDATE + 7,
           '09:00',
           'PROGRAMADA'
    FROM paciente
    WHERE ROWNUM <= 3;
    DBMS_OUTPUT.PUT_LINE('La sentencia inserto ' || SQL%ROWCOUNT
        || ' citas de una sola vez.');
    COMMIT;
END;
/
```